# Project – Low Level Design

# On

# Dockerized Sports Rust Service

Course Name: DevOps

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1. | Himani Jaiswal | EN22CS301423 |
| 2. | Irya Patni | EN22CS301436 |
| 3. | Uday Dubey | EN22CS301058 |
| 4. | Chetan Oswal | EN22C301295 |

*Group Name:BTechGroup07*

*Project Number:DO-50*

*Industry Mentor Name:*

*University Mentor Name:*

*Academic Year:2025-2026*

# Contents

1. Introduction.
   1.1. Scope
   1.2. Intended Audience
   1.3.System Design

2. System Design.
   2.1. Application Design
   2.2. Process Flow
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

# 1. Introduction

The Dockerized Sports Rust Service is a backend system developed to manage sports-related data efficiently using modern software engineering and DevOps practices. The system is implemented using the Rust programming language and deployed using Docker containers to ensure consistency across development and production environments.

In traditional application development, inconsistencies between environments often lead to deployment failures. This project solves such issues by introducing containerization, which encapsulates the application along with its dependencies into a single portable unit.

Rust is chosen as the core language due to:

- Memory safety without garbage collection
- High performance comparable to C/C++
- Strong concurrency support

Docker is used to:

- Package the application
- Maintain consistent runtime environments
- Simplify deployment

The system exposes RESTful APIs to perform CRUD operations on sports entities such as teams, players, and matches. It is designed with modular architecture principles to ensure scalability, maintainability, and reliability.

# 2. Scope of the Document

This document provides a detailed low-level design of the Dockerized Sports Rust Service. It explains internal system components, their interactions, and deployment strategies.

### In Scope

- Backend system design and architecture
- Application structure and module breakdown
- Database schema and relationships
- REST API design and endpoints
- Dockerfile and containerization strategy

- Docker Compose orchestration
- Process flow and data flow
- Component-level design
- Key design considerations

### Out of Scope

- Frontend/UI development
- Cloud-native orchestration (Kubernetes)
- Authentication and authorization mechanisms (advanced)
- Third-party API integrations

## 3. Intended Audience

This document is intended for the following stakeholders:

### 1. Developers

- Backend developers working with Rust
- Contributors implementing APIs and business logic

### 2. DevOps Engineers

- Responsible for containerization and deployment
- Managing Docker configurations and environments

### 3. System Architects

- Reviewing system structure and scalability
- Evaluating architectural decisions

### 4. Academic Evaluators

- Faculty assessing the project
- Reviewing understanding of system design and DevOps

### Required Knowledge

- Basic understanding of REST APIs
- Familiarity with Docker and containers
- Knowledge of relational databases

# 4. System Overview

The system is a containerized backend service that provides APIs for managing sports-related data.

### Key Functionalities

- Create, update, delete, and retrieve teams
- Manage players and assign them to teams
- Record and track matches
- Provide structured JSON responses

### System Components

1. **Rust API Service**
    a. Implements business logic
    b. Handles client requests
2. **PostgreSQL Database**
    a. Stores structured data
    b. Maintains relationships
3. **Docker Engine**
    a. Builds and runs containers
4. **Docker Compose**
    a. Orchestrates multiple services

### System Characteristics

- Stateless application design
- Persistent database storage
- Container-based deployment
- Platform-independent execution

# 5. System Design

The system follows a **layered architecture pattern** to ensure separation of concerns and modularity.

### Architecture Layers

#### *1. API Layer (Presentation Layer)*

- Exposes REST endpoints
- Handles HTTP requests and responses
- Performs input validation

#### *2. Service Layer (Business Logic)*

- Contains core application logic
- Applies validation and transformations

#### *3. Repository Layer (Data Access)*

- Interacts with database
- Executes queries

#### *4. Database Layer*

- Stores persistent data
- Maintains constraints and relationships

### Design Principles

- **Separation of Concerns:** Each layer has a distinct responsibility
- **Loose Coupling:** Components interact via defined interfaces
- **High Cohesion:** Related functionalities are grouped together
- **Scalability:** Supports horizontal scaling via containers

## 6. Application Design

The application is designed using a modular structure for better maintainability.

### Project Structure

```
src/
 ├── main.rs
 ├── routes/
 ├── handlers/
 ├── services/
```

```
├── repository/
├── models/
└── config/
```

### Module Description

- **main.rs**
  - Entry point
  - Initializes server and middleware
- **routes/**
  - Defines API endpoints
- **handlers/**
  - Processes incoming requests
- **services/**
  - Implements business logic
- **repository/**
  - Handles database operations
- **models/**
  - Defines data structures
- **config/**
  - Manages environment variables

### Design Approach

- Clean architecture
- Modular codebase
- Reusable components

## 7. Process Flow

The process flow describes how the system processes requests.

### Detailed Flow Steps

1. Client sends HTTP request
2. Request reaches API endpoint
3. Handler validates request

4. Service layer applies logic
5. Repository executes query
6. Database returns data
7. Response sent to client

### Flow Representation

```
Client → API → Handler → Service → Repository → Database → Response
```

### Observations

- Clear separation between layers
- Easy debugging and testing
- Supports scalability

## 8. Information Flow

Information flow describes how data moves across the system.

### Input Flow

- Client sends JSON request
- Request parsed into Rust structures

### Processing Flow

- Validation checks applied
- Business rules executed
- Data transformed

### Output Flow

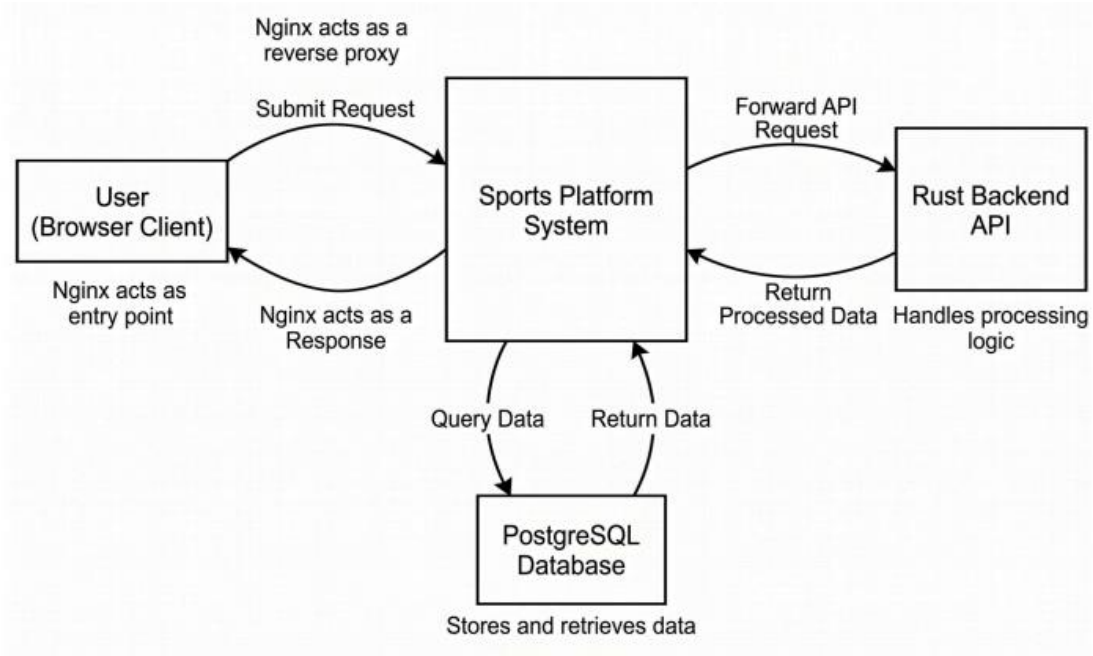
- Data retrieved from DB
- Converted into JSON
- Sent as response

**Input:**

```
{
  "name": "Team A"
}
```

**Output:**

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "Team A"
  }
}
```




## 9. Components Design (Detailed)

The system is designed using a **layered architecture**, where each component has a specific responsibility. This separation improves scalability, maintainability, and clarity.

## 1. API Component

The API component acts as the **entry point** of the application.

- Handles all incoming HTTP requests from clients (frontend or external systems)
- Defines **routes/endpoints** such as `/teams`, `/players`, `/matches`
- Validates request data (e.g., required fields, formats)
- Converts incoming requests into appropriate service calls
- Sends structured responses (JSON format) back to the client

**Example Responsibilities:**

- Routing (`GET /teams`, `POST /players`)
- Input validation
- Response formatting

**Importance:**
Ensures smooth communication between the client and backend while maintaining a clean interface.

## 2. Service Component

The Service component contains the **core business logic** of the application.

- Processes data received from the API layer
- Applies business rules (e.g., validating team existence before adding a player)
- Coordinates between multiple components (API ↔ Repository)
- Handles error logic and decision-making

**Example Responsibilities:**

- Checking if a team exists before assigning a player
- Managing match scheduling logic
- Transforming raw data into meaningful results

**Importance:**
Keeps business logic separate from request handling, making the system easier to maintain and scale.

## 3. Repository Component

The Repository component manages all interactions with the database.

- Executes SQL queries (INSERT, SELECT, UPDATE, DELETE)
- Provides an abstraction layer over database operations
- Converts database rows into application-friendly objects

**Example Responsibilities:**

- Fetching all teams from database
- Saving new player records
- Querying match data

**Importance:**
Decouples database logic from business logic, allowing easy database changes in the future.

## 4. Database Component

The database component uses PostgreSQL to store and manage persistent data.

- Stores structured data such as teams, players, and matches
- Ensures **data integrity** using constraints and relationships
- Supports transactions (ACID properties)
- Provides indexing for faster query performance

**Example Tables:**

- Teams (id, name, coach)
- Players (id, name, team_id, role)
- Matches (id, team1_id, team2_id, date)

**Importance:**
Acts as the backbone of the application by securely storing all critical data.

## 5. Docker Component

The Docker component uses Docker to containerize the application.

- Packages the application along with its dependencies
- Ensures consistency across different environments
- Simplifies deployment and scaling

**Key Features:**

- Isolated runtime environment
- Lightweight containers
- Easy version control of environments

**Importance:**
Eliminates environment-related issues and improves deployment reliability.

## 6. Docker Compose Component

The Docker Compose component uses Docker Compose to manage multiple services.

- Defines services (backend, database) in a single configuration file (`docker-compose.yml`)
- Handles **service orchestration**
- Manages networking between containers
- Allows starting all services with a single command

**Example Services:**

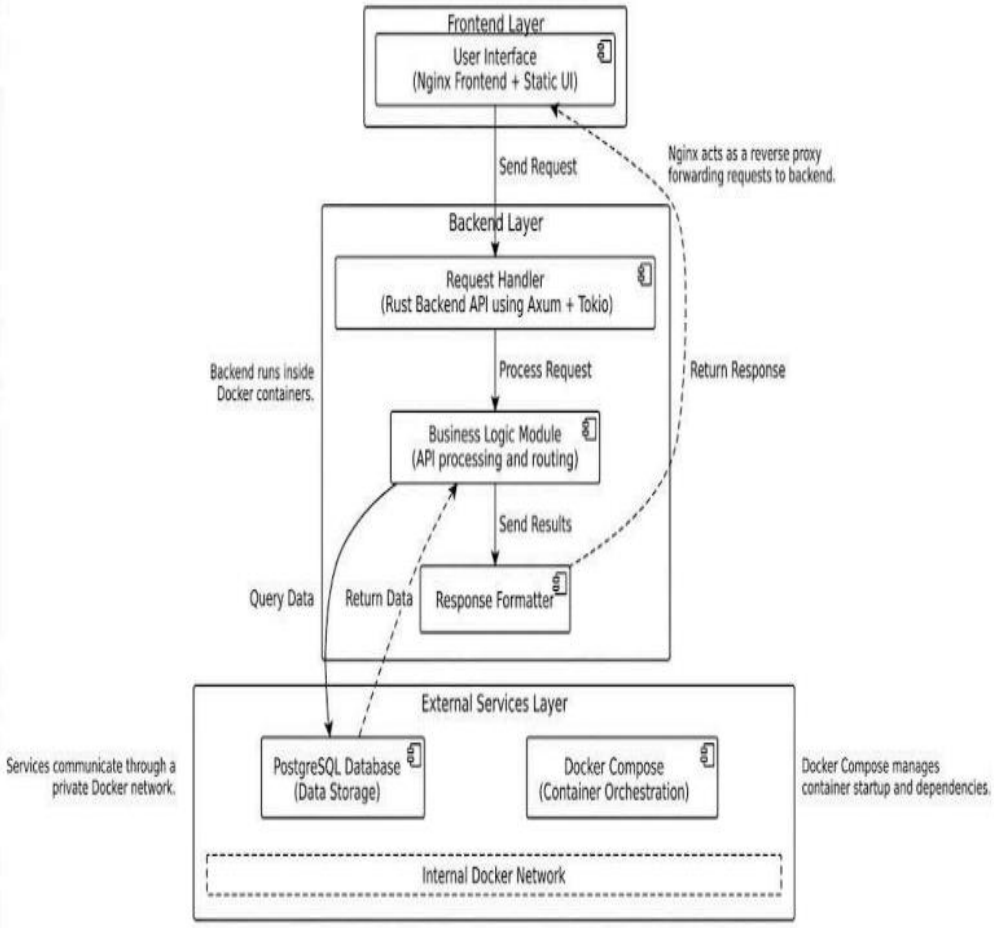
- Backend service (Rust API)
- Database service (PostgreSQL)

**Importance:**
Simplifies multi-container application management and improves developer productivity.

## Overall Architecture Flow

1. Client sends request → API Component
2. API forwards request → Service Component
3. Service processes logic → Repository Component
4. Repository interacts → Database

5. Response flows back to client




# 10. Key Design Considerations (Detailed)

## 1. Performance

Performance is a critical factor in backend system design. Using Rust ensures:

- **High execution speed** comparable to C/C++
- **Zero-cost abstractions**, meaning no runtime overhead
- **Efficient memory management** without a garbage collector

This results in faster API responses and better handling of concurrent requests, especially under heavy load.

## 2. Scalability

The system is designed to scale efficiently using Docker.

- **Container-based scaling:** Multiple instances of the service can run simultaneously
- **Horizontal scaling:** Easily deploy additional containers based on traffic
- **Stateless architecture:** Each request is independent, allowing load balancing across instances

This ensures the application can handle increasing users without major architectural changes.

## 3. Security

Security is integrated at multiple levels:

- **Minimal base images:** Reduces attack surface by including only necessary dependencies
- **Non-root containers:** Prevents privilege escalation attacks
- **Secure environment variables:** Sensitive data (API keys, DB credentials) is not hardcoded

These practices help protect against common vulnerabilities in containerized applications.

## 4. Portability

Using Docker makes the application highly portable:

- Runs consistently across **development, testing, and production environments**
- Eliminates "works on my machine" issues
- Compatible with cloud platforms and local systems

## 5. Reliability

Reliability ensures continuous availability:

- **Restart policies:** Containers automatically restart on failure
- **Persistent storage:** Important data is stored outside containers (e.g., volumes)
- **Fault isolation:** Failure in one container does not crash the entire system

### 6. Maintainability

The system follows clean architecture principles:

- **Modular design:** Code is divided into reusable modules
- **Separation of concerns:** API, business logic, and database layers are independent
- Easier debugging, testing, and future updates

### 7. Image Optimization

Optimizing Docker images improves efficiency:

- **Multi-stage builds:** Separate build and runtime environments
- **Reduced image size:** Removes unnecessary dependencies
- Faster deployment and lower storage usage

## 11. API Catalogue

### Base URL

http://localhost:8080

This is the root endpoint where all API requests are directed in a local development environment.

# 1. Team APIs

## GET /teams

- **Purpose:** Fetch all teams from the database
- **Method:** GET
- **Response:** List of teams

**Example Response:**

```
{
  "status": "success",
  "data": [
    { "id": 1, "name": "Team A" },
    { "id": 2, "name": "Team B" }
  ]
}
```

## POST /teams

- **Purpose:** Create a new team
- **Method:** POST
- **Request Body:**

```
{
  "name": "Team A",
  "coach": "John Doe"
}
```

- **Response:**

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "Team A"
  }
}
```

## 2. Player APIs

### GET /players

- **Purpose:** Retrieve all players
- **Method:** GET
- **Response:** List of players

### POST /players

- **Purpose:** Add a new player
- **Method:** POST
- **Request Example:**

```
{
  "name": "Player 1",
  "team_id": 1,
  "role": "Batsman"
}
```

- **Response Example:**

```
{
  "status": "success",
  "data": {
    "id": 10,
    "name": "Player 1"
  }
}
```

## 3. Match APIs

### GET /matches

- **Purpose:** Retrieve match details
- **Method:** GET
- **Response:** List of matches

### POST /matches

- **Purpose:** Create a new match
- **Method:** POST
- **Request Example:**

```
{
  "team1_id": 1,
  "team2_id": 2,
  "date": "2026-04-22"
}
```

- **Response Example:**

```
{
  "status": "success",
  "data": {
    "match_id": 101
  }
}
```

## API Design Principles Used

The API structure follows best practices of REST API Design:

- **Resource-based endpoints** (`/teams`, `/players`, `/matches`)
- **Proper HTTP methods** (GET, POST)
- **Consistent response format**
- **Stateless communication**

## 12. References

### 1. Docker Official Documentation

- **Website:** https://docs.docker.com
- **Description:**
  The official documentation for Docker provides comprehensive guidance on containerization, including installation, image creation, container management, networking, and volumes.
- **Key Topics Used:**
  - Creating Docker images using Dockerfile
  - Container lifecycle (build, run, stop)
  - Networking between services
  - Volume management for persistent data

- **Why Important:**
  Docker enables consistent environments across development, testing, and production, which is critical for scalable applications.

## 2. Rust Programming Language Documentation

- **Website:** https://www.rust-lang.org/learn
- **Description:**
  Official Rust documentation includes "The Rust Book," API references, and tutorials covering ownership, borrowing, concurrency, and memory safety.
- **Key Topics Used:**
  - Memory safety without garbage collection
  - Concurrency handling
  - Error handling (`Result`, `Option`)
- **Why Important:**
  Rust ensures high performance and safety, making it ideal for backend services requiring reliability.

## 3. PostgreSQL Documentation

- **Website:** https://www.postgresql.org/docs/
- **Description:**
  PostgreSQL documentation provides in-depth knowledge about database management, SQL queries, indexing, transactions, and performance optimization.
- **Key Topics Used:**
  - Schema design and normalization
  - CRUD operations
  - Indexing and query optimization
  - ACID compliance
- **Why Important:**
  PostgreSQL is a powerful, open-source relational database widely used in production systems.

## 4. Docker Compose Documentation

- **Website:** https://docs.docker.com/compose/
- **Description:**
  Docker Compose documentation explains how to define and manage multi-container applications using a `docker-compose.yml` file.
- **Key Topics Used:**
  - Service configuration
  - Multi-container orchestration
  - Environment variables and networking
- **Why Important:**
  Simplifies running complex applications with multiple services (backend, database, etc.) in a single command.

## 5. REST API Design Guidelines

- **Description:**
  REST API design guidelines define best practices for building scalable and maintainable APIs using HTTP methods.
- **Key Topics Used:**
  - HTTP methods (GET, POST, PUT, DELETE)
  - Status codes (200, 404, 500, etc.)
  - Resource-based URL design
  - Stateless communication
- **Why Important:**
  Ensures clean, consistent, and scalable communication between frontend and backend systems.